

Documentação do Código do Robô UAILL-E

Lucas Lemos Ricaldoni

Março
2025

Sumário

1	Introdução	4
2	Dependências	4
3	Estrutura do Código	5
4	Explicação das Funcionalidades	5
4.1	line_detection.py	5
4.1.1	calculate_error()	5
4.1.2	identify_colour()	6
4.1.3	verify_curve	8
4.1.4	detect_line	9
4.1.5	Exemplo de utilização	10
4.2	ball_detection.py	11
4.2.1	identify_colour	11
4.2.2	find_ball	12
4.2.3	Exemplo de utilização	14
4.3	main.py	14
4.3.1	Função Principal	14
4.3.2	Inicialização da Câmera	16
4.3.3	Verificação de Obstáculos e Linha	17
4.3.4	Ajuste de Movimento e Controle do Robô	17
4.4	robot_control.py	18

4.4.1	Inicialização e Configuração	18
4.4.2	Carregamento de Bibliotecas em C	19
4.4.3	Controle de Movimentação	19
4.4.4	Desvio de Obstáculos	20
4.4.5	Medição de Distância	20
4.4.6	Resgate de Objetos	21
4.5	PID.c	21
4.6	motors.c	22
4.6.1	Inicialização dos motores	22
4.6.2	Controle de estado do motor	22
4.6.3	Controle das direções dos motores	23
4.6.4	Movimento de curva de 90	24
5	Referências Bibliográficas	25
5.1	OpenCV	25
5.2	Thresholding (Binarização)	25
5.3	Detecção de bordas	26
5.4	Operações Morfológicas	26
5.4.1	Opening (Abertura)	27
5.4.2	Opening (Closing)	27
5.5	Segmentação de cor	28
5.5.1	Hue Saturation Value (HSV)	28
5.6	Transformada de Hough	29

5.7	Controle PID	30
5.8	Ctypes	31
6	Conclusão	31

1 Introdução

Este documento apresenta a documentação do código desenvolvido para o projeto de um **seguidor de linha** da **categoria avançada** na modalidade **Resgate** da **Olimpíada Brasileira de Robótica 2025**.

A lógica de detecção da linha na pista e dos desafios é baseada em conceitos de **visão computacional**. A detecção é realizada a partir de uma **câmera**, que utiliza algoritmos de visão computacional, como **detecção de bordas**, combinados com lógica de programação para permitir que o robô percorra corretamente o caminho.

Além disso, na fase de **resgate**, o sistema é projetado para **detectar bolas**, identificando suas **posições** e **cores**, garantindo que o robô consiga cumprir sua missão com precisão.

2 Dependências

O código foi desenvolvido em **Python** e em **C**. Para o desenvolvimento do sistema utilizamos as seguintes bibliotecas:

- NumPy
- OpenCV
- Matplotlib
- Ctypes
- time
- pigpio

Os modelos de visão computacional foram desenvolvidos a partir de testes realizados no ambiente de **Jupyter Notebooks**.

3 Estrutura do Código

O projeto está organizado da seguinte forma:

```
1  OBR2025/  
2  |-- c_files/  
3  |   |-- motors.c  
4  |   |-- motors.h  
5  |   |-- PID.c  
6  |   |-- PID.h  
7  |-- datas/  
8  |   |-- lines/  
9  |   |-- balls/  
10 |-- notebooks/  
11 |   |-- line-detection.ipnyb  
12 |   |-- model-to-detect-balls.ipnyb  
13 |-- pdfs/  
14 |   |-- line-detection.pdf  
15 |   |-- model-to-detect-balls.pdf  
16 |   |-- documentation.pdf  
17 |-- main.py  
18 |-- robot_control.py  
19 |-- line_detection.py  
20 |-- ball_detection.py
```

4 Explicação das Funcionalidades

Cada função possui sua docstring que facilita o entendimento do que ela realiza. Abaixo terá as explicações das principais funções e suas utilizações. Além da explicação de que cada arquivo impacta no robô.

4.1 line_detection.py

Este arquivo é responsável pela detecção das linhas, apartir de algoritmos de visao computacional que possibilita a detecção das linhas. Abaixo terá as funções presentes no arquivo.

4.1.1 calculate_error()

```

1 def calculate_error(target_line):
2     if isinstance(target_line, tuple):
3         target_line = target_line[0]
4
5     # Convert to numpy array e remove the extra dimensions
6     contour = np.squeeze(np.array(target_line))
7
8     # avoid erro if has few points
9     if contour.shape[0] < 2:
10        return None
11
12    # get the extrames points
13    x1, y1 = tuple(contour[np.argmin(contour[:, 1])]) # few
14    value of Y (top)
15    x2, y2 = tuple(contour[np.argmax(contour[:, 1])]) #
16    bigger value of Y (base)
17
18    # calculate angle em rad
19    theta_rad = np.arctan2(y2 - y1, x2 - x1)
20
21    # convert to degree
22    theta_deg = np.degrees(theta_rad)
23
24    # assure the degrees in [0, 180]
25    theta_deg = theta_deg + 180 if theta_deg < 0 else
26    theta_deg
27
28    # return the erro in degree (for less use of memory)
29    return int(theta_deg - 90)

```

Função 1: Cálculo do erro.

Esse trecho de código é utilizado para calcular o erro do robô em relação a linha. O erro é calculado a partir da inclinação do robô em relação a linha de referência (target_line). Tal erro é retornado em graus.

4.1.2 identify_colour()

```

1 def identify_colour(img):
2     # convert image in HSV scale
3     hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
4
5     # define colour ranges
6     color_ranges = {
7         "red": [(np.array([0, 100, 100]), np.array([10, 255,
8         255])),
9         (np.array([160, 100, 100]), np.array([180,
10        255, 255]))], # Red (Hue 0-10)

```

```

9         "green": [(np.array([40, 40, 40]), np.array([90, 255,
10            255]))], # Green (Hue 40-90)
11         "gray": [(np.array([0, 0, 50]), np.array([130, 60,
12            220]))] # Gray
13     }
14
15     kernel = np.ones((3, 3), np.uint8)
16     # create a dict to storage the colours
17     detected_areas = {}
18
19     # find colours
20     for color, ranges in color_ranges.items():
21         # create mask colours
22         mask = sum(cv2.inRange(hsv_img, lower, upper) for
23 lower, upper in ranges)
24         # reduce the noisy
25         mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
26
27         contours, _ = cv2.findContours(mask, cv2.
28 RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
29         detected_areas[color] = sum(cv2.contourArea(c) for c
30 in contours)
31
32     if contours:
33         # drawn the contours in black colour
34         cv2.drawContours(img, contours, -1, (0,0,0), 2)
35
36     if max(detected_areas.values()) > 0:
37         dominant_color = max(detected_areas, key=
38 detected_areas.get)
39
40         if dominant_color == 'green':
41             # split the image in two sides
42             mid = img.shape[1] // 2
43
44             # create a green mask
45             green_mask = sum(cv2.inRange(hsv_img, lower,
46 upper) for lower, upper in color_ranges['green'])
47
48             # calculate moments of image
49             moments = cv2.moments(green_mask)
50
51             # find the center of object
52             cX = int(moments["m10"] / moments["m00"])
53
54             # find the side
55             side = 'left' if cX < mid else 'right'
56
57             # return the green colour and your side

```

```

51         return dominant_color, side
52
53         # return the colour with most area on the image
54         return dominant_color, None
55     # return none if not has colour in the image
56     return None

```

Função 2: Identificação de cores.

Este trecho de código é responsável pela detecção das cores presentes na pista, excluindo o preto e branco, focando em cores como vermelho, verde e cinza, que determinam o trajeto a ser seguido. Com base em uma escala de cores previamente definida, é possível utilizar "máscaras" para separar as áreas da imagem que correspondem a essas cores específicas.

Além de realizar a segmentação de cores, o código inclui funções de visualização que desenham as áreas identificadas com as cores selecionadas, facilitando a análise do processo. O retorno pode ser `None` caso nenhuma cor relevante seja detectada, ou a cor identificada, juntamente com sua posição na pista, caso uma correspondência seja encontrada.

Para o funcionamento da mesma, é preciso enviar a imagem como parâmetro.

4.1.3 verify_curve

```

1 def verify_curve(contours, img_widht):
2     # Split the image in 2
3     mid = img_width // 2
4
5     for contour in contours:
6         X = contour[:, 0, 0]
7
8         points_left, points_right = np.sum(X < mid), np.sum(X
9         >= mid)
10
11         # if contour > 8 probably it's not a curve 90
12         if len(contour) < 8:
13             if points_left > points_right:
14                 return 'Left'
15             if points_left < points_right:
16                 return 'Right'
17
18     # return no curve
19     return False

```

Função 3: Verifica curva de 90°

A função acima tem o objetivo de detectar se há uma interseção de 90° na pista que não está identificada por um quadrado verde. A interseção pode ser para a direita ou esquerda.

A lógica da detecção é baseada na divisão da imagem ao longo do eixo x, separando-a em dois lados (esquerdo e direito). A partir disso, calcula-se qual lado possui a maior quantidade de pontos, retornando o lado que apresenta uma distribuição maior de pontos do que o outro, indicando uma curva de 90°. Caso não haja curva, o retorno é falso.

4.1.4 detect_line

```
1 def detect_line(img):
2     # verify if img exists
3     if img is not None:
4         # Convert the image to grayscale
5         img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
6
7         # Aplly a threshold to detect only darken colours
8         _, binary = cv2.threshold(img_gray, 50, 255, cv2.
THRESH_BINARY_INV)
9
10        # reduce the noise
11        kernel = np.ones((3, 3), np.uint8)
12        binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN,
kernel)
13        binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE,
kernel)
14
15        # Detect points that form a line
16        contours, _ = cv2.findContours(binary, cv2.
RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
17
18        # verify if has a symbol on the img
19        cor_detected = identify_colour(img)
20
21        # verify if has a line on the image
22        if len(contours) > 0:
23            # drawn the line target
24            cv2.drawContours(img, contours, -1, (0, 0, 255),
2)
25
26            #draw the car flow line
27            draw_line(img)
28
```

```

29         # send img_widht as img.shape[1]
30         is_curve = verify_curve(contours, img.shape[1])
31
32         # calculate the error
33         erro = calculate_error(contours)
34
35         # to avoid false-positive
36         if (erro > 0 and erro <= 10) or (erro >= -10 and
erro < 0): is_curve = False
37
38         # to avoid false-positive
39         if cor_detected or is_curve != False: return 0,
is_curve, cor_detected
40
41         # return the erro and is_curve
42         return erro, is_curve, cor_detected
43
44     return None, None, None

```

Função 4: Detecta a linha

A função utiliza algoritmos de visão computacional para detecção de bordas (*edge detection*) e identificação de formas (*Hough Transform*). O processo inclui a conversão da imagem para a escala de cinza, seguida da aplicação de filtros para redução de ruídos. O código executa uma lógica que detecta o número de contornos na imagem, permitindo analisar a posição da linha presente.

Essa função é a principal responsável pela lógica de **detecção de linha**. Ela identifica possíveis cores na imagem, determina as posições e formatos das linhas, detecta a presença de curvas de 90° e calcula o erro do robô em relação ao trajeto.

A função recebe apenas a imagem como parâmetro e retorna no formato de uma tupla: (erro, possibilidade_de_curva, cor_detectada). Caso uma das variáveis não seja identificado, a função retornará apenas ele como **None**.

4.1.5 Exemplo de utilização

```

1 if __name__ == '__main__':
2     path = 'datas/lines/'
3     images = [f for f in os.listdir(path) if f.endswith('.jpg
')]
4

```

```

5     # for a specified file
6     #images = ["ladrinho_embranco.jpg"]
7
8     for image_name in images:
9         print(path + image_name)
10        img = cv2.imread(path + image_name)
11        #img = cv2.imread(image_name)
12
13        erro, is_curve, has_colour = detect_line(img)
14        print(erro, is_curve, has_colour)
15
16        image_name, type_file = image_name.split('.jpg')
17        download_image(erro, path, image_name, is_curve)
18        print(f"Memory used: {psutil.Process(os.getpid()).
memory_info().rss / (1024 ** 2)} MB")
19        print('-'*35)

```

Função 5: Exemplo de utilização

Se você ainda não entendeu como a lógica funciona, teste o código acima para compreendê-la melhor. O código extrai todas as imagens de teste contendo linhas e as envia para a função *detect_line()* para realizar a análise completa da imagem.

Esse trecho imprime no terminal o **erro**, a **existência de uma curva** e se há uma **cor diferente** na imagem. Além disso, o código permite visualizar o uso de memória gasto durante a execução de cada imagem.

4.2 ball_detection.py

Este arquivo foi desenvolvido para a detecção das cores e posições das bolas na arena de resgate, usando algoritmos de visão computacional. Abaixo terá as funções presentes no arquivo.

4.2.1 identify_colour

```

1     def identify_colour(hsv_colour):
2         h, s, v = hsv_colour
3
4         if v < 50:
5             return "Black"
6         elif s < 50 and 50 <= v < 200:
7             return "Grey"
8         elif s < 50 and v >= 200:
9             return "White"

```

```

10         elif (0 <= h <= 10) or (170 <= h <= 180):
11             return "Red"
12         elif 25 <= h <= 35:
13             return "Yellow"
14         elif 35 <= h <= 85:
15             return "Green"
16         elif 100 <= h <= 130:
17             return "Blue"
18         else:
19             return "Unknown"

```

Função 6: Identifica cores

Esse trecho a partir de uma análise de uma imagem na escala **HSV** *Hue* (*matriz*), *Saturation* (*saturação*) e *Value* (*valor*) retorna a cor dominante a partir dos valores da matriz.

Caso ainda tenha uma duvida, consulte as referências bibliográficas deste pdf.

4.2.2 find_ball

```

1 def find_ball(img):
2     # resize the image into dimensions ESP32-cam
3     desired_width = 320
4     desired_height = 240
5     dim = (desired_width, desired_height)
6     img = cv2.resize(img, dim, interpolation=cv2.INTER_AREA)
7
8     # to debug
9     #print(img.shape)
10
11     # transform the img in gray-scale
12     img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
13
14     # applying filtes in the image
15     img_gray = cv2.medianBlur(img_gray, 5)
16
17     # use the function to found circles on the image
18     circles = cv2.HoughCircles(img_gray, cv2.HOUGH_GRADIENT,
19                                dp=1.5, minDist=40,
20                                param1=60, param2=30, minRadius=5,
21                                maxRadius=60)
22
23     # verify if has circles on the image
24     if circles is not None:
25         circles = np.uint16(np.around(circles))

```

```

24         for i in circles[0,:]:
25             # cordenates of the circle
26             x, y, r = i[0], i[1], i[2]
27
28             # draw the outer circle
29             cv2.circle(img,(i[0],i[1]),i[2],(0,255,0),2)
30             # draw the center of the circle
31             center_position = (i[0], i[1])
32             cv2.circle(img,(i[0],i[1]),2,(0,0,255),3)
33
34             # change the image in HSV
35             img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
36
37             # Extracting the mean color
38             mask = np.zeros(img_hsv.shape[:2], dtype=np.uint8
39 )
40             # create the mask
41             cv2.circle(mask, (x, y), r, 255, thickness=-1)
42
43             # calculate the medium color
44             mean_colour_hsv = cv2.mean(img_hsv, mask=mask)
45             [:3]
46
47             # get the color of the object
48             img_colour = identify_colour(mean_colour_hsv)
49
50             text_position = (x - 10, y - r - 10)
51             # writing the colour on the image
52             cv2.putText(img, img_colour, text_position,
53                 fontFace=cv2.FONT_HERSHEY_SIMPLEX,
54                 fontScale=0.6,
55                 color=(0, 255, 0), thickness=1,
56                 lineType=cv2.LINE_AA)
57             return img_colour, center_position
58         return None, None

```

Função 7: Detecta a bola

O código começa redimensionando a imagem para garantir que ela esteja na melhor escala possível. Em seguida, a imagem é convertida para a escala de cinza e um filtro é aplicado para reduzir o ruído.

A detecção dos círculos (bolas) é realizada por meio da função *HoughCircles* do OpenCV. A partir dos círculos identificados, são armazenadas as coordenadas (x, y) do centro de cada um

A detecção de cor é feita convertendo a imagem para a escala **HSV** e calculando a média dos valores nessa escala. Para fins de melhor visualização,

os círculos detectados são desenhados na imagem, e um texto indicando a respectiva cor é adicionado.

A função retorna uma tupla no formato (cor_da_bola, posição_da_bola)

4.2.3 Exemplo de utilização

```
1 if __name__ == '__main__':
2     img = cv2.imread('datas/balls/one-yellow-ball.jpg')
3     ball_colour, ball_position = find_ball(img)
4     print(ball_colour, ball_position)
```

Função 8: Exemplo de utilização

Se você ainda não entendeu como a lógica funciona, teste o código acima para compreendê-la melhor. O código lê uma imagem do banco de dados e a envia para a função *find_ball()* para realizar a análise completa da imagem.

Esse trecho imprime no terminal a **cor da bola**, a **posição da bola**.

4.3 main.py

O arquivo `main.py` tem como objetivo gerenciar todas as funcionalidades do robô, coordenando a comunicação entre os diferentes módulos do sistema. Ele importa os demais arquivos escritos em Python e implementa a lógica central para o funcionamento adequado do robô.

4.3.1 Função Principal

```
1 def main():
2     # define the constat values
3     Kp = 150; Ki = 0; Kd = 0; previous_erro = 0; PID = 0
4     right_velocity_curve = 200; left_velocity_curve = 200
5
6     # init the cam
7     s = 'camera-path'
8     cam = cv2.VideoCapture(s)
9
10    # loop to read the cam
11    while cv2.waitKey(1) != 27:
12        # extract the cam info
13        has_frame, img = cam.read()
14
15        # verify if the cam working
```

```

16         if not has_frame:
17             break
18
19         # verify if has an object on front
20         distance = measure_distance()
21         if distance is not None and distance <= MAX_DISTANCE:
22             avoid_obstacle()
23
24         # calculate the error
25         erro, is_curve, has_colour = detect_line(img)
26
27         # if not has line it try to come back of line
28         if erro is None:
29             motors.run_backward(right_velocity_curve,
30             left_velocity_curve)
31             time.sleep(0.5)
32             motors.stop_motor()
33
34         # I still have to decided the magic numbers
35         if has_colour is not None:
36             colour, side_curve = has_colour
37
38             # verify if is going to rescue area
39             if colour == 'grey':
40                 rescue_area()
41
42             # verify if has a 90 degrees curve
43             elif colour == 'green':
44                 # turn on the correct side
45                 if side_curve == 'left':
46                     motors.turn_right(right_velocity_curve,
47                     left_velocity_curve)
48                 elif side_curve == 'right':
49                     motors.turn_left(right_velocity_curve,
50                     left_velocity_curve)
51
52             elif colour == 'red':
53                 # stop the car on the red line
54                 motors.stop()
55                 # ALL it's run fine
56                 break
57
58         # verify if has a curve
59         if is_curve is not None:
60             if is_curve is 'left':
61                 motors.turn_left(right_velocity_curve,
62                 left_velocity_curve)
63             elif is_curve is 'right':

```

```

60         motors.turn_right(right_velocity_curve,
left_velocity_curve)
61     else:
62         # calculate PID
63         PID = PID_functions.calculate_PID(erro,
previous_erro, Kp, Kd, Ki)
64         previous_erro = erro
65
66         # adjust move the car
67         adjust_move(PID)
68
69     # restart the cam memory
70     cam.release()

```

Função 9: Função Principal

O código acima é responsável pelo o gerenciamento de todas as funções. Abaixo tem a explicação de cada trecho da função acima.

```

1 def main():
2     # Defini o de constantes
3     Kp = 150; Ki = 0; Kd = 0; previous_erro = 0; PID = 0
4     right_velocity_curve = 200; left_velocity_curve = 200

```

Função 10: Definição das variáveis do PID

Esse trecho inicializa as variáveis utilizadas no cálculo do PID (Proporcional-Integral-Derivativo), fundamentais para o controle de trajetória do robô. Os valores podem ser ajustados conforme testes e calibrações realizadas.

4.3.2 Inicialização da Câmera

```

1     # Inicializa o da c mera
2     s = 'camera-path'
3     cam = cv2.VideoCapture(s)
4
5     # Loop para captura de imagens
6     while cv2.waitKey(1) != 27:
7         # Captura de quadro da c mera
8         has_frame, img = cam.read()
9
10        # Verifica o do funcionamento da c mera
11        if not has_frame:
12            break

```

Função 11: Inicialização da câmera

Neste trecho, a câmera é inicializada utilizando a biblioteca OpenCV. O loop garante a captura contínua de imagens, enquanto a estrutura condicional verifica se a câmera está operando corretamente no Raspberry Pi.

4.3.3 Verificação de Obstáculos e Linha

```
1  # Verifica o de obstaculos
2  distance = measure_distance()
3  if distance is not None and distance <= MAX_DISTANCE:
4      avoid_obstacle()
5
6  # C lculo do erro baseado na linha
7  erro, is_curve, has_colour = detect_line(img)
```

Função 12: Coleta de dados do ambiente

Esse trecho assegura que o robô obtenha dados sobre seu percurso. Primeiro, o sensor ultrassônico mede a distância de obstáculos à frente. Em seguida, a função `detect_line()` analisa a posição do robô em relação à linha de referência.

4.3.4 Ajuste de Movimento e Controle do Robô

```
1  # Se n o houver linha detectada, tenta retornar ao
   trajeto
2  if erro is None:
3      motors.run_backward(right_velocity_curve,
4      left_velocity_curve)
5      time.sleep(0.5)
6      motors.stop_motor()
7
8  # Decis o baseada na cor detectada
9  if has_colour is not None:
10     colour, side_curve = has_colour
11
12     if colour == 'grey':
13         rescue_area()
14     elif colour == 'green':
15         if side_curve == 'left':
16             motors.turn_right(right_velocity_curve,
17             left_velocity_curve)
18         elif side_curve == 'right':
19             motors.turn_left(right_velocity_curve,
20             left_velocity_curve)
21     elif colour == 'red':
22         motors.stop()
```

```

20         break
21
22     # Verifica o de curvas
23     if is_curve is not None:
24         if is_curve == 'left':
25             motors.turn_left(right_velocity_curve,
26 left_velocity_curve)
27         elif is_curve == 'right':
28             motors.turn_right(right_velocity_curve,
29 left_velocity_curve)
30         else:
31             # C lculo do PID
32             PID = PID_functions.calculate_PID(erro,
33 previous_erro, Kp, Kd, Ki)
34             previous_erro = erro
35
36             # Ajuste do movimento do rob
37             adjust_move(PID)
38
39 # Libera o da c mera
40 cam.release()

```

Função 13: Ajuste do movimento do robô

Este trecho implementa a lógica de movimentação do robô com base nos dados coletados. Caso o robô perca a linha de referência, ele executa uma manobra para tentar reencontrá-la. Além disso, a detecção de cores permite tomar decisões específicas, como identificar curvas de 90° (verde), áreas de resgate (cinza) ou paradas obrigatórias (vermelho). Quando não há curvas detectadas, o controle PID é utilizado para manter o robô sobre a trajetória desejada.

4.4 robot_control.py

O arquivo acima foi desenvolvido em Python para o controle de um robô baseado em Raspberry Pi. Ele inclui a integração com sensores ultrassônicos, motores e um controlador PID, utilizando bibliotecas como *pigpio* e *ctypes* para comunicação com componentes de hardware.

4.4.1 Inicialização e Configuração

```

1 import pigpio
2 import ctypes
3 import time
4 from ball_detection import *

```

```

5
6 # initialize pigpio
7 pi = pigpio.pi()
8
9 # define the magic number
10 MAX_DISTANCE = 8
11
12 # define Pins
13 TRIG = 9
14 ECHO = 10
15 # set pins
16 pi.set_mode(TRIG, pigpio.OUTPUT)
17 pi.set_mode(ECHO, pigpio.INPUT)

```

Função 14: Inicialização e configuração dos pinos

O código começa inicializando o módulo *pigpio* e definindo as constantes e pinos utilizados.

4.4.2 Carregamento de Bibliotecas em C

```

1 # load C files
2 PID_functions = ctypes.CDLL("./c_files/PID.so")
3 motors = ctypes.CDLL("./c_files/motors.so")
4
5 # Define argumentos e retornos das funções
6 PID_functions.calculate_PID.argtypes = (ctypes.c_int, ctypes.c_int, ctypes.c_int, ctypes.c_int)
7 PID_functions.calculate_PID.restype = ctypes.c_int
8
9 motors.run.argtypes = [ctypes.c_int, ctypes.c_int]
10 motors.run_backward.argtypes = [ctypes.c_int, ctypes.c_int]
11 motors.turn_right.argtypes = [ctypes.c_int, ctypes.c_int]
12 motors.turn_left.argtypes = [ctypes.c_int, ctypes.c_int]
13 motors.stop_motor.argtypes = []

```

Função 15: Carregamento das funções em C

O código carrega bibliotecas escritas em C para controle PID e motores, tornando a execução mais eficiente em tempo real.

4.4.3 Controle de Movimentação

```

1 def adjust_move(PID):
2     base_right_velocity = 180
3     base_left_velocity = 180
4

```

```

5     right_velocity = max(0, min(base_right_velocity - PID,
6     left_velocity = max(0, min(base_left_velocity + PID, 255)
7     )
8     motors.run(right_velocity, left_velocity)

```

Função 16: Ajuste da velocidade dos motores

A função *adjust_move(PID)* ajusta a velocidade dos motores com base na saída do controlador PID.

4.4.4 Desvio de Obstáculos

```

1 def avoid_obstacle():
2     right_velocity = 255; left_velocity = 255
3     motors.stop()
4     time.sleep(1)
5     motors.run(right_velocity, left_velocity/2)
6     time.sleep(2)
7     motors.run(right_velocity, left_velocity)
8     time.sleep(4)
9     motors.run(right_velocity/2, left_velocity)
10    time.sleep(1)
11    motors.run(right_velocity, left_velocity)
12    time.sleep(8)
13    motors.run(right_velocity/2, left_velocity)
14    time.sleep(1)
15    motors.run(right_velocity, left_velocity)
16    time.sleep(4)

```

Função 17: Desvio de obstáculos

A função *avoid_obstacle()* executa uma sequência de movimentos para evitar colisões utilizando o sensor ultrassônico.

4.4.5 Medição de Distância

```

1 def measure_distance():
2     pi.write(TRIG, 1)
3     time.sleep(0.00001)
4     pi.write(TRIG, 0)
5
6     while pi.read(ECHO) == 0:
7         start = time.time()
8
9     while pi.read(ECHO) == 1:
10        end = time.time()

```

```

11
12     duration = end - start
13     distance = (duration * 34300) / 2
14     return distance

```

Função 18: Medição de distância com sensor ultrassônico

A função *measure_distance()* mede a distância até o objeto mais próximo usando um sensor ultrassônico.

4.4.6 Resgate de Objetos

```

1 def rescue_area(img):
2     ball_colour, ball_position = find_ball(img)
3     # precisa fazer com que o carro siga a posi o da bola

```

Função 19: Detecção e resgate de objetos

A função *rescue_area(img)* é responsável por detectar e seguir objetos, como bolas, mas ainda não foi implementada completamente.

4.5 PID.c

O arquivo acima foi desenvolvido em C para o calculo do PID (Proportional Integral Derivativo) de uma maneira mais eficiente e otimizada que em Python.

```

1 int calculate_PID(int erro, int previous_erro, int Kp, int Kd
  , int Ki) {
2     // define param values
3     int PID = 0;
4     int P = erro;
5     static int I = 0;
6     // limit the I on -255:255
7     I += P;
8     if (I > 255) {I = 255;}
9     if (I < -255) {I = -255;}
10
11     int D = erro - previous_erro;
12
13     // calculate the PID
14     PID = (Kp * P) + (Ki * I) + (Kd * D);
15
16     // atualiza o valor do PID
17     return PID;
18 }

```

Função 20: Implementação do controle PID em C

O código acima implementa o cálculo do PID utilizando a seguinte equação matemática:

$$PID(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (1)$$

A utilização do PID permite um ajuste preciso e rápido do robô, garantindo uma resposta eficiente as variações da pista.

4.6 motors.c

O arquivo acima foi desenvolvido em C para o ajuste de velocidade e estados dos motores, com um tempo de resposta baixo.

4.6.1 Inicialização dos motores

```
1 // define motors ports
2 const int MOTOR_LEFT_CLKWISE = 3;
3 const int MOTOR_LEFT_ANTI = 4;
4 const int MOTOR_RIGHT_CLKWISE = 7;
5 const int MOTOR_RIGHT_ANTI = 8;
6
7 const int MOTOR_PWM_LEFT = 5;
8 const int MOTOR_PWM_RIGHT = 6;
9
10 void setup_motor() {
11     gpioInitialise();
12     gpioSetMode(MOTOR_LEFT_CLKWISE, PI_OUTPUT);
13     gpioSetMode(MOTOR_LEFT_ANTI, PI_OUTPUT);
14     gpioSetMode(MOTOR_RIGHT_CLKWISE, PI_OUTPUT);
15     gpioSetMode(MOTOR_RIGHT_ANTI, PI_OUTPUT);
16
17     gpioSetMode(MOTOR_PWM_LEFT, PI_OUTPUT);
18     gpioSetMode(MOTOR_PWM_RIGHT, PI_OUTPUT);
19 }
```

Função 21: Setup motores

Este trecho inicializa os motores, configurando as portas digitais e PWM, utilizando a biblioteca **gpio** a partir da função `gpioSetMode()`

4.6.2 Controle de estado do motor

```
1 void set_state_motor(int leftCw, int leftCcw, int rightCw,
2     int rightCcw) {
3     gpioWrite(MOTOR_LEFT_ANTI, leftCcw);
```

```

3  gpioWrite(MOTOR_LEFT_CLKWISE, leftCw);
4  gpioWrite(MOTOR_RIGHT_ANTI, rightCcw);
5  gpioWrite(MOTOR_RIGHT_CLKWISE, rightCw);
6  }

```

Função 22: Controle do estado dos motores

Este trecho utiliza os estados dos motores para controlar a direção que os motores irão girar. A tabela abaixo especifica qual variável controla o estado de cada motor. O valor **HIGH** ativa o motor, enquanto **LOW** o desliga.

Variavel	Descrição
leftCw	Motor Esquerdo Horário
leftCcw	Motor Esquerdo Anti-Horário
rightCw	Motor Direito Horário
rightCcw	Motor Direito Anti-Horário

Table 1: Descrição das variáveis de controle dos motores

4.6.3 Controle das direções dos motores

```

1  void run(int velocityRight, int velocityLeft) {
2      // set motor states
3      set_state_motor(HIGH, LOW, HIGH, LOW);
4      // set motors velocity
5      gpioPWM(MOTOR_PWM_LEFT, velocityLeft);
6      gpioPWM(MOTOR_PWM_RIGHT, velocityRight);
7  }
8
9  void stop_motor() {
10     // set motor states
11     set_state_motor(LOW, LOW, LOW, LOW);
12 }
13
14 void run_backward(int velocityRight, int velocityLeft) {
15     // set motor states
16     set_state_motor(LOW, HIGH, LOW, HIGH);
17     //set motor velocity
18     gpioPWM(MOTOR_PWM_LEFT, velocityLeft);
19     gpioPWM(MOTOR_PWM_RIGHT, velocityRight);
20 }

```

Função 23: Controle da direção dos motores

As funções acima são responsáveis pelo controle da direção dos movimentos que o robô irá fazer. A tabela abaixo explica detalhadamente as funcionalidades da função.

Função	Descrição
run	Controla a direção (frente, esquerda, direita)
stop_motor	Desliga os motores
run_backward	Liga o motor no sentido anti-horario

Table 2: Descrição das funções de controle dos motores e seus efeitos sobre o movimento

Os valor *HIGH*(1) representa o sentido ligado e o *LOW*(0) no sentido desligado

4.6.4 Movimento de curva de 90

```

1 void turn_left(int velocityRight, int velocityLeft) {
2     //set motor states
3     set_state_motor(HIGH, LOW, LOW, HIGH);
4     //set motor velocity
5     gpioPWM(MOTOR_PWM_LEFT, velocityLeft);
6     gpioPWM(MOTOR_PWM_RIGHT, velocityRight + 40);
7     usleep(DELAY_TIME);
8 }
9
10 void turn_right(int velocityRight, int velocityLeft) {
11     // set motor states
12     set_state_motor(LOW, HIGH, HIGH, LOW);
13     //set motor velocity
14     gpioPWM(MOTOR_PWM_LEFT, velocityLeft + 40);
15     gpioPWM(MOTOR_PWM_RIGHT, velocityRight);
16     usleep(DELAY_TIME);
17 }

```

Função 24: Funções para rotação de 90° à esquerda e à direita

As funções acima realizam curvas de 90° utilizando a lógica de chamada `turn_{lado_escolhido}` (em inglês, `turn_left` ou `turn_right`). Durante a curva, o motor do lado oposto ao giro recebe um aumento de velocidade de +40 unidades, auxiliando na rotação mais eficiente.

5 Referências Bibliográficas

5.1 OpenCV

O OpenCV (Open Computer Vision) é uma biblioteca open-source de python destinada a visão computacional. Ela reúne as principais funcionalidades do mundo da computação inteligente baseada no tratamento de imagens. Facilitando ao acesso da visão computacional, democratizando a análise de imagens e videos. A escolha do OpenCV nesse projeto se deve a sua capacidade de fornecer as nossas demandas de forma eficiente.

5.2 Thresholding (Binarização)

O Thresholding é utilizado para binarizar a imagem.

O que seria binarizar algo?

Um valor binário varia entre apenas dois valores: 0 ou 1, que nesse caso representam preto e branco. A binarização funciona a partir da aproximação do valor do pixel na escala RGB ao preto (0) ou ao branco (255).

Por exemplo, se o valor de Px_1 for 120, ele será classificado como preto, pois está mais próximo de 0. Já se $Px_2 = 190$, esse pixel será binarizado como branco, pois está mais próximo de 255.

Caso ainda tenha dúvidas, a imagem abaixo pode ajudar a entender melhor:

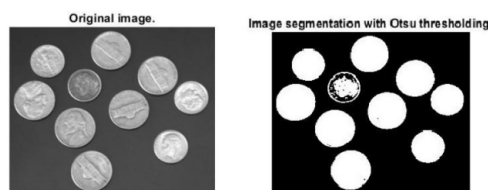


Figure 1: Imagem aplicada ao filtro Thresholding

Mas porque aplicar o efeito thresholding?

A binarização por Thresholding permite destacar os contornos e formas dos objetos na imagem, tornando a segmentação mais precisa. Como podemos ver na imagem acima, essa técnica é útil para identificar e destacar os elementos importantes na imagem.

Na competição, essa prática é essencial para a identificação das linhas na pista, pois permite destacar e segmentar claramente o formato da trajetória a ser seguida.

5.3 Detecção de bordas

A detecção de bordas é um método utilizado na visão computacional para separar pontos da imagem nos quais ocorre uma diferença significativa de intensidade (ou iluminação). Essa técnica é essencial para destacar os contornos dos objetos presentes na imagem, sendo fundamental para tarefas como segmentação e reconhecimento de formas.

No código do robô foi utilizada a função `findContours`, do OpenCV, para identificar os contornos dos objetos. Essa função trabalha em conjunto com algoritmos de detecção de bordas (como Canny) e retorna uma lista de contornos detectados, que auxiliam na identificação de linhas pela pista.

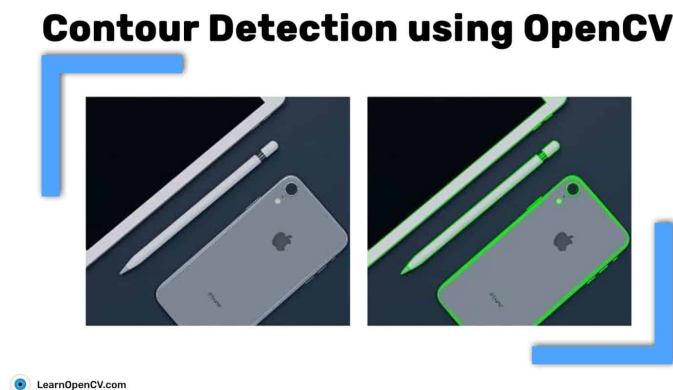


Figure 2: Exemplo da aplicação do Find Contours

5.4 Operações Morfológicas

As *operações morfológicas* são um conjunto de técnicas usadas no processamento de imagens com o objetivo de analisar a forma e a estrutura de objetos dentro de uma imagem. Usadas principalmente para remover o ruído e extrair formas em uma imagem.

Essas operações seguem o formato de matriz para realizar seus cálculos, abaixo há uma imagem exemplificando como esse algoritmo detecta um quadrado, retângulo e um círculo.

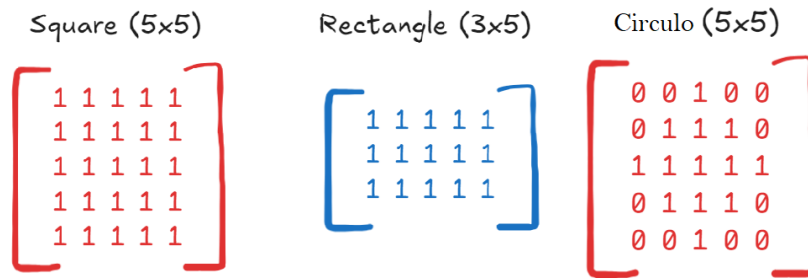


Figure 3: Formato matricial (quadrado, retangulo e circulo)

As operações usadas no desenvolvimento do robô foram **Opening (Abertura)** e **Closing (Fechamento)**.

5.4.1 Opening (Abertura)

A operação de abertura é usada para remover pequenos objetos da imagem (ruídos), preservando as formas de objetos mais significativos.

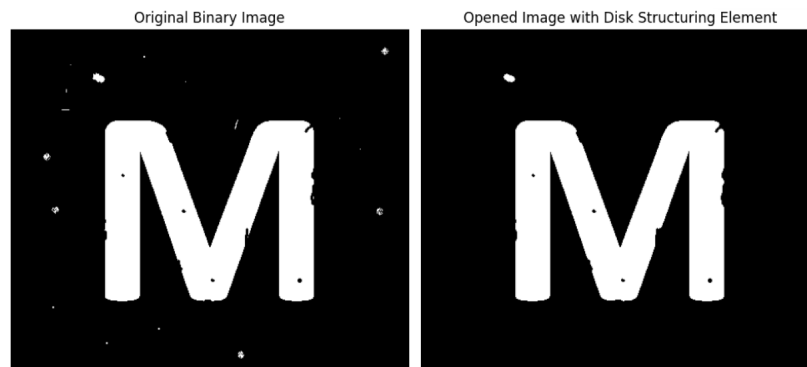


Figure 4: Operação de Abertura

5.4.2 Opening (Closing)

A operação de fechamento é o inverso da operação de abertura, usada para fechar pequenos buracos dentro dos objetos. Resumindo, sua funcionalidade é suavizar os contornos dos objetos na imagem.

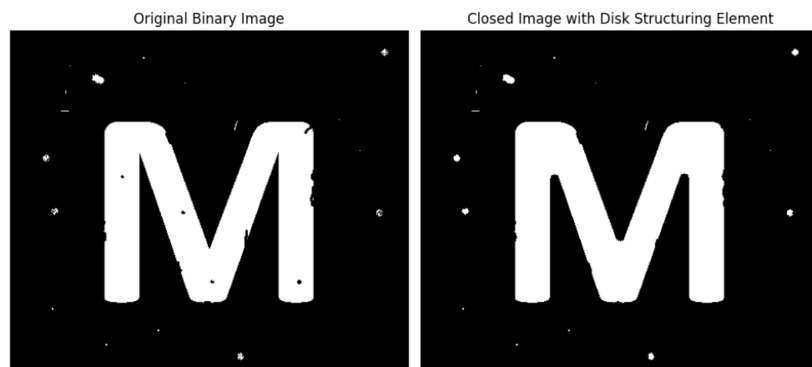


Figure 5: Operação de Fechamento

Ao aplicar os dois métodos juntos, conseguimos eliminar os ruídos na imagem e realçar o objeto em destaque.

5.5 Segmentação de cor

Para a detecção de cores na imagem usamos a segmentação de cor, que consiste na transformação da imagem para a escala HSV (está explicada logo abaixo). Para detectar as cores é necessário determinar um range mínimo e máximo da cor desejada na escala HSV. Com isso, ao aplicar esse range pela imagem criamos mascaras em torno dos objetos que estão dentro desses limites.



Figure 6: Processo de Segmentação de cor

5.5.1 Hue Saturation Value (HSV)

A escala Hue Saturation Value, possui o H (hue) que representa a tonalidade da cor (variando de 0 a 179), o S (saturation) é a saturação/intensidade da

cor (variando de 0 a 255) e o V (value) é o brilho da cor (variando de 0 a 255). Abaixo há uma comparação da escala HSV com a escala RGB.

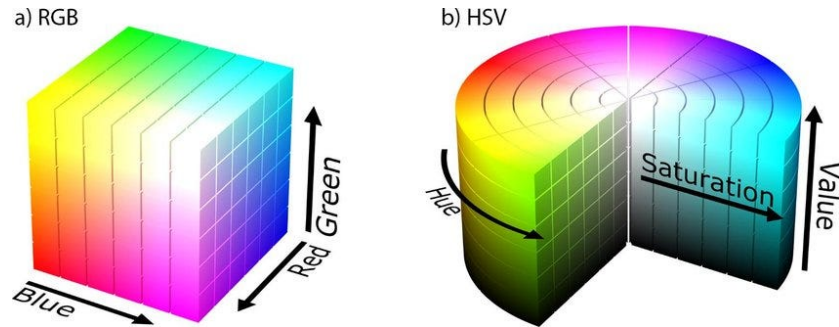


Figure 7: Escala RGB vs HSV

É possível ver que a escala RGB é um cubo tridimensional, onde cada dimensão corresponde a intensidade das cores. Na escala HSV é um formato circular, mais próximo da percepção humana. Para projetos de segmentação e detecção a escala HSV é a melhor, pois ela permite isolar tonalidades(H), saturação(S) e brilho(V) de forma mais clara e eficiente.

5.6 Transformada de Hough

A transformação de Hough é usada para a identificação de formas geométricas em imagens, como círculos e retas. O princípio de funcionamento é baseado na transformação dos pontos da imagem em espaço de parâmetros. Facilitando a detecção delas. A aplicação dela no carro foi para a detecção de círculos, partindo da definição da circunferência:

$$r^2 = (x - x_c)^2 + (y - y_c)^2 \quad (2)$$

O que essa equação nos diz?

Ela demonstra que um círculo é definido por 2 parâmetros: seu raio r e pelas coordenadas do seu centro (x_c, y_c) . Essa transformação separa os possíveis círculos da imagem, que depois da aplicação do **Canny Edge Detection**, o método de Hough separa os principais candidatos a círculos na imagem. Para definir quais objetos são círculos na imagem, usamos um acumulador tridimensional (x_c, y_c, r) onde cada célula acumula "indicações". Após percorrer por toda a imagem os picos no acumulador retornam os círculos presentes.

5.7 Controle PID

O Controle PID (Proporcional Integral Derivativo) é usado para o controle de processos. O seu objetivo é apartir de equações matemáticas que envolvem integrais e derivadas, minimizar o erro do sistema. O controle proporcional ajusta a variável de controle de forma proporcional ao erro. O controle integral ajusta a variável de controle baseando-se no tempo em que o erro acontece. O controle derivativo ajusta a variável de controle tendo como base a taxa de variação do erro.

Equação do controle PID:

$$PID(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (3)$$

Nessa equação, temos os seguintes parâmetros:

- K_p : ganho proporcional;
- K_i : ganho integral;
- K_d : ganho derivativo;
- e : erro instantâneo (diferença entre o valor desejado e o valor medido);
- τ : variável de integração;
- t : tempo atual.

Abaixo há uma tabela explicando como se deve ajustar os parâmetros do PID conforme o erro que o robô apresenta.

Comportamento do Robô	Parâmetro a Ajustar	Ação Recomendada
Oscila muito em torno da linha (zig-zag)	Proporcional (K_p)	Diminua o valor de K_p . Um K_p muito alto causa reações exageradas.
Corrige devagar e não consegue seguir bem a linha	Proporcional (K_p)	Aumente o valor de K_p . Um K_p muito baixo gera correções fracas.
Erro pequeno persiste por muito tempo	Integral (K_i)	Aumente K_i para corrigir erros acumulados lentamente.
Começa a oscilar após um tempo seguindo bem	Integral (K_i)	Diminua K_i . Um K_i alto pode acumular erro demais e causar instabilidade.
Reação muito abrupta a mudanças rápidas na linha	Derivativo (K_d)	Aumente K_d para suavizar a resposta. Ele ajuda a "frear" mudanças bruscas.
Resposta lenta a mudanças rápidas	Derivativo (K_d)	Diminua K_d se o robô demorar demais para reagir a curvas ou desvios.

Table 3: Ajuste dos parâmetros do PID com base no comportamento do robô

5.8 Ctypes

Ctypes é uma biblioteca que permite a comunicação de arquivos desenvolvidos em C. A linguagem C por ser uma linguagem de baixo nível (linguagem que está próxima do hardware do computador) permite uma eficiência enorme em termos de cálculos e manipulação de memória comparado a linguagens de alto nível como o Python. Porém o Python é uma linguagem muito versátil, onde temos fácil acesso a bibliotecas de visão computacional complexas.

Para continuar usando a praticidade do Python e ter um desempenho em partes que exigem uma resposta rápida a utilização do Ctypes é essencial.

O ctypes funciona lendo um arquivo (.SO), a biblioteca carrega esse arquivo e permite a utilização completa do código escrito em C usando a sintaxe do Python, unindo a praticidade e a eficiência.

6 Conclusão

Este documento apresenta a estrutura e as funcionalidades do código, facilitando o entendimento do sistema e promovendo a democratização do mesmo. Ele proporciona liberdade e autonomia para a realização de testes e a utilização do código em diferentes contextos, permitindo uma maior flexibilidade no desenvolvimento e aprimoramento do projeto.